东 南 大 学 考 试 卷（2015 秋）

1. 说说随机算法和近似算法的相似之处与不同之处。（7 分）

不同之处

特性	随机算法	近似算法
定义	利用随机性引导算法运行过程，解的质量可能依赖随机过程（如随机选择路径、变量或初始状态）。	设计一个确定性算法，找到接近最优解的结果，通常通过分析确定解与最优解之间的比值（近似比）。
随机性	本质是**非确定性**算法，解的结果与随机过程密切相关，可能在不同运行中产生不同解。	本质是**确定性**算法，结果固定，和随机性无关。
质量保障	随机算法一般只能通过**概率分析**保证得到较优解的概率较高，但无法保证找到一个固定误差范围内的解。	近似算法有**严格的近似比分析**，能够保证解在最优解的某个固定倍数范围内，如$(1+\epsilon)\cdot OPT$。
运行时间	运行时间可能依赖随机过程，有时需要多次运行随机算法以提高解的质量。	运行时间通常是固定的，因为近似算法是确定性的。
典型问题	**蒙特卡洛算法**、**拉斯维加斯算法**（如随机快速排序、随机化图匹配）。	**贪心算法**、**启发式方法**（如旅行商问题、顶点覆盖问题的近似解）。
应用领域	在需要探索大规模解空间的场景中，例如机器学习中的随机梯度下降、图论问题中的随机路径搜索等。	在需要对解的误差有明确控制的场景中，例如背包问题、图分割问题等，特别是实际需要性能保证的工程应用。
是否依赖多次运行	可能需要多次运行以得到较优解，解的质量提高依赖运行次数。	只需运行一次即可得到解，解的质量是算法设计固有的特性。
复杂度分析	复杂度往往通过**期望时间**或**成功概率**来衡量，例如某随机算法的期望时间复杂度为 $O(f(n))$。	复杂度直接由算法步骤分析，例如时间复杂度 $O(f(n))$，并提供一个固定的近似比，如解接近最优解的 1.5 倍。
结果的重复性	结果可能因随机性而不同，需要多次运行后选取较优解。	结果具有确定性，同样输入必然输出同样结果。

相似之处

1. **目标一致**：
   - **随机算法**和**近似算法**都用于解决复杂问题，尤其是对于**NP 难问题**或大规模问题，通常无法在多项式时间内找到精确解。
   - 两者都试图在**时间复杂度**和**解的质量**之间寻找平衡。
2. **非精确性**：
   - 两种算法通常都无法保证找到问题的最优解，而是提供一个可接受的解。
   - 它们在实际应用中往往表现为**启发式**方法，能够找到满足要求的“较优解”。
3. **应用场景重叠**：
   - 两种方法都常用于优化问题，如**背包问题**、**图论问题**（如最小割、最大流等）和**机器学习中的搜索问题**。

2. 说说你对 NP 完全性的理解。（8 分）

一个问题被称为 **NP 完全问题**，需要满足以下两个条件：

1. **问题属于 NP 类**：即，存在一个**多项式时间验证算法**来验证其解的正确性。
2. **问题是 NP 困难（NP-hard）**：该问题是 NP 类中最难的问题。具体来说，**所有 NP 类问题都可以在多项式时间内规约到该问题**。

- **NP 完全性**是计算复杂性理论中的一个核心概念，描述了一类问题的难度。如果一个 NP 完全问题能在多项式时间内被解决，那么所有 NP 类问题都能在多项式时间内解决。
- **P 与 NP 问题**的关系尚未解决：
- 如果 $P = NP$ ，那么所有 NP 类问题都可以高效地解决。
- 如果 $P \neq NP$ ，则 NP 完全问题无法在多项式时间内求解。

NP 完全问题的特性

1. **普遍性**：

- 很多实际问题，如**旅行商问题**（TSP）、**布尔可满足性问题**（SAT）、**图的顶点覆盖问题**等，都是 NP 完全问题。
- 这些问题出现在众多领域中（如优化、人工智能、图像处理等）。

2. **多项式时间规约**：

- A \leq_p B ：问题 A 可以被转换为问题 B ，并且转换过程在多项式时间内完成。
- 如果一个问题可以规约到另一个 NP 完全问题，那么该问题至少与 NP 完全问题一样困难。

3. **求解的两种方式：**

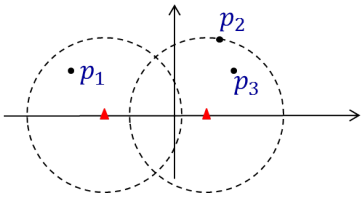
- **暴力枚举**：枚举所有可能的解，时间复杂度通常是指数级别。
- **近似算法或启发式方法**：在实际应用中，尝试找到近似解来减少计算开销。

经典 NP 完全问题

以下是一些著名的 NP 完全问题：

1. **布尔可满足性问题（SAT）**：给定一个布尔公式，判断是否存在一种变量赋值使公式为真。
2. **旅行商问题（TSP）**：给定一组城市和两两之间的距离，求一个访问每个城市仅一次的最短路径。
3. **顶点覆盖问题**：在图中找到一个最小的顶点集合，使得图中每条边至少有一个端点在集合中。
4. **子集和问题**：给定一个整数集合，判断是否存在一个子集，其和等于某个目标值。

3. 海面上有一些船需要与陆地进行通信，需要在海岸线上布置一些基站。现将问题抽象为，在 x 轴上方，给出 N 条船的坐标$p_1, p_2, \ldots, p_N$，$p_i = (x_i, y_i)$，$x_i \geq 0, y_i \leq \mathrm{d}, 1 \leq i \leq N$，在 x 轴上安放的基站可以覆盖半径为 d 的区域内的所有点，问在 x 轴上至少要安放几个点才可以将 x 轴上方的点都覆盖起来。试设计一个算法求解该问题，并分析算法的正确性。（20 分）



算法设计思路

1. **确定每条船的覆盖区间**：
   - 对于每艘船 $p_i = (x_i, y_i)$，基站必须位于其下方 x_i 的垂直线的覆盖范围内。
   - 基站的覆盖半径为 d，因此船 p_i 的覆盖范围可以表示为区间 $[x_i - \sqrt{d^2 - y_i^2}, x_i + \sqrt{d^2 - y_i^2}]$。
   - 如果 $y_i > d$，说明该船无法被覆盖，输出“不可能覆盖”。
2. **将所有覆盖区间按区间的右端点排序**：
   - 按区间的右端点从小到大排序，这样有助于用贪心算法选择基站的位置。
3. **使用贪心算法覆盖所有区间**：
   - 从左到右扫描所有区间，每次选择当前覆盖到的最右端点作为基站位置，然后跳过所有被当前基站覆盖的区间。
   - 记录基站的数量。

```
Input: N个船的坐标 (x_1, y_1), ..., (x_N, y_N), 基站覆盖半径 d
Output: 最少的基站数量

1. 初始化一个空列表 intervals
2. 遍历每艘船的坐标 (x_i, y_i):
    a. 如果 y_i > d:
         输出 "不可能覆盖" 并退出
    b. 计算覆盖区间 [x_i - sqrt(d^2 - y_i^2), x_i + sqrt(d^2 - y_i^2)]
    c. 将该区间加入 intervals

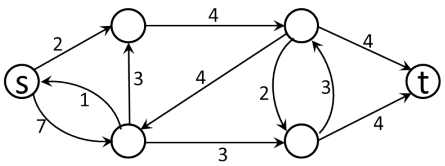
3. 将 intervals 按照区间右端点从小到大排序

4. 初始化基站数量 count = 0, 当前位置 current_end = -∞

5. 遍历 intervals 中的每个区间 [start, end]:
    a. 如果当前区间的左端点 start > current_end:
         i. 在当前区间的右端点 end 处放置一个基站
         ii. 更新 current_end = end
         iii. count += 1

6. 输出 count
```

4. 下图是一个流网络，请说明 s 到 t 的最大流值是多少？并标示一种流方式。（15 分）



最大流值是 7

5. 有一实数序列$a_1, a_2, \ldots, a_N$，若$i < j$ 且 $a_i > a_j$，则(a_i, a_j)构成了一个逆序对，请使用分治方法求整个序列中逆序对个数，并分析算法的时间复杂性。（20 分）

例如：序列(4,3,2)逆序对有(4,3)，(4,2)，(3,2)共 3 个。

使用**分治法**解决逆序对计数问题，其核心思想与**归并排序**类似：

1. **分解子问题：**

- 将序列分为两部分，分别为左半部分 L 和右半部分 R 。
- 分别统计 L 和 R 内部的逆序对数量，以及跨越 L 和 R 的逆序对数量。

2. **合并子问题的结果：**

- 合并 L 和 R 时，统计跨越左右部分的逆序对数量，并保持 L 和 R 合并后的顺序是递增的。
- 通过双指针的方法，在合并时统计逆序对。

3. **递归处理：**

- 对每个子序列递归分治，直到子序列长度为 1，此时逆序对数量为 0。

```
function countInversions(arr, left, right):
    if left >= right:
        return 0

    // Step 1: 分解子问题
    mid = (left + right) // 2
    leftCount = countInversions(arr, left, mid)
    rightCount = countInversions(arr, mid + 1, right)

    // Step 2: 统计跨越左右部分的逆序对并合并
    crossCount = mergeAndCount(arr, left, mid, right)

    // Step 3: 合并结果
    return leftCount + rightCount + crossCount

function mergeAndCount(arr, left, mid, right):
    temp = []
    i = left, j = mid + 1
    count = 0

    // Merge左右两部分并统计逆序对
    while i <= mid and j <= right:
        if arr[i] <= arr[j]:
            temp.append(arr[i])
            i += 1
        else:
            temp.append(arr[j])
            j += 1
            count += (mid - i + 1) // 左侧未合并元素数目

    // 处理剩余元素
    while i <= mid:
        temp.append(arr[i])
        i += 1

    while j <= right:
        temp.append(arr[j])
        j += 1

    // 拷贝临时数组回原数组
    for k in range(len(temp)):
        arr[left + k] = temp[k]

    return count

// 主函数
function inversionCount(arr):
    return countInversions(arr, 0, len(arr) - 1)
```

6. 给定一长度为 N 的整数序列 $a_1, a_2, \ldots, a_N$，将其划分成多个子序列，满足每个子序列中整数的和不大于一个数 B，设计一种划分方法，最小化所有子序列中最大值的和。说明其具有优化子结构性质及子问题重叠性质。（20 分）

例如：序列长度为 8 的整数序列(2,2,2,8,1,8,2,1)，B=17，可将其划分成三个子序列(2,2,2)，(8,1,8)以及(2,1)，则可满足每个子序列中整数和不大于 17，所有子序列中最大值的和 12 为最终结果。

思想

1. 定义状态：

令 $dp[i]$ 表示前 i 个元素划分的最优解（所有子序列中最大值的和的最小值）。

2. 转移方程：

$$dp[i] = \min_{j<i} \left(\max(dp[j], \text{sum}(a[j+1..i])) \right)$$

- j 表示划分点。
- $\text{sum}(a[j+1..i])$ 表示从第 $j+1$ 个元素到第 i 个元素的和。
- 当前最优值取 $dp[j]$ 和 $\text{sum}(a[j+1..i])$ 中较大的值。

3. 边界条件：

- $dp[0] = 0$：没有元素时，不需要划分，结果为 0。

4. 优化子结构：

- 将子序列划分问题转化为求解最优分割点问题。

```
function minimizeMaxSum(a, N, B):
    dp = [∞] * (N + 1)
    dp[0] = 0

    for i = 1 to N:
        totalSum = 0
        for j = i to 1 step -1:
            totalSum += a[j]
            if totalSum > B:
                break
            dp[i] = min(dp[i], max(dp[j - 1], totalSum))

    return dp[N]
```

优化子结构与子问题重叠

1. **优化子结构：**

当前问题的最优解依赖于前缀的最优划分，因此子问题可以独立求解。

2. **子问题重叠：**

对于同一前缀，划分方案可能被多次求解，通过动态规划或二分法减少冗余计算。

7. 考虑这样一个应用问题，市场上销售的某型钢材的长度为 1，现需要一些长度分别为$a_1, a_2, \ldots, a_N$的小钢条（$a_i \leq 1, 1 \leq i \leq N$），问至少要买多少根钢条进行切割？请给出一算法，使得买的钢条尽可能少，并分析该算法的近似比。（10 分）

装箱问题是 NP 难问题，因此无法找到高效的精确算法。常见的启发式算法如下：

1. 贪心算法：首次适配（First Fit Decreasing, FFD）

1. **步骤：**

- 按照需求值 $a_1, a_2, \ldots, a_N$ 从大到小排序。
- 初始化空的钢条集合。
- 遍历需求值 a_i，尝试放入已有钢条中，若不能放入，则新开一根钢条。

```
function minSteelBars(a, N):
    sort(a, descending)  // 按需求从大到小排序
    bins = []            // 初始化空的钢条集合

    for i in range(1, N+1):
        placed = False
        for j in range(1, len(bins) + 1):
            if bins[j] + a[i] <= 1:
                bins[j] += a[i]  // 放入已有钢条
                placed = True
                break
        if not placed:
            bins.append(a[i])    // 新开一根钢条

    return len(bins)  // 返回钢条数量
```

近似比分析

FFD 是装箱问题的常用近似算法，其近似比为 1.22 （理论上已证明）。

- **定义：**

设最优解（最少钢条数）为 OPT ，FFD 结果为 ALG 。则：

\frac{ALG}{OPT} \leq 1.22